# Act It Out, Then Bind: Verifier-in-the-Loop Compilation of Natural-Language Requests into Reusable Robot Skills

Frenchies
Zürich

**Abstract.** A robot skill marketplace has to turn a customer's sentence ("mix me a smoothie") into a sequence of purchasable skills that a robot can actually execute, and say precisely what is missing when it cannot. We present SKILLSTORE, which compiles requests in two stages: a language model first *acts out* the request as atomic hand movements without seeing the catalog, then *binds* those movements to catalog skills under a schema that only admits real skill identifiers. A symbolic simulator over hand-scoped STRIPS-style predicates, object kinds and a contents ledger checks every plan and returns its complaints to the model for repair. Uncovered movements become atomic, reusable *gaps*, each matched to an open demonstration dataset, and unserved gaps are priced as bids for data. On 155 compilations of 13 requests by four planners (GPT-4.1 mini, GPT-4.1, Claude Sonnet 5.5, Claude Opus 5.5), 18% of first bindings are rejected by the simulator and 3% remain rejected after two repair rounds; with feedback, mid-sized planners match the largest at a fraction of its latency. Most first-binding failures are unmet preconditions, but some pass every precondition check and are caught only by object kinds and the contents ledger. Searching hand assignments shortens the resulting two-arm schedules by 29%. 11% of plan steps are gaps; nearly all map to an open dataset, yet mostly to human egocentric video rather than robot demonstrations, and their names must be grouped before demand can be priced.

## 1 Introduction

Robot skills are becoming products: an operator buys *grasp bottle* or *wipe surface* for a specific arm, and expects them to compose into a process. Vision-language-action policies [2, 3, 16, 21] make such skills cheaper to train from a few dozen demonstrations, and open corpora [4, 15, 22] make the demonstrations easier to find. What remains is the interface. A customer does not shop for `move_to_station`; they ask for a smoothie, a clean table or a painted picture, and expect the store to work out which skills that takes, which of them exist, and what is missing.

Natural language makes a language model the obvious compiler—and an unreliable one. Language models produce plans that read well but are physically incoherent: pouring before grasping, closing a bottle grasp around a bucket, holding a blender jar while pressing its start button, or pouring a glass into the blender instead of the other way round [14, 24]. For a store this is worse than an error message, because an incoherent plan is also a wrong bill, a wrong installation on a physical arm, and a wrong estimate of what data to collect next.

We take the position of LLM-Modulo frameworks [14]: the model proposes, a sound checker decides. Our contributions are:

- a **two-stage compiler** that separates *what a person's hands would do* from *which product does it*, so that uncovered work surfaces as atomic, reusable gaps instead of being dropped or collapsed into one opaque "draw image with brush" step (§3.2);
- a **symbolic simulator** whose checks are pure functions of declared data—skill pre/postconditions, the object kinds a skill acts on, and a ledger of what each container holds—with no task-specific rules (§3.3);
- an **episode-level scheduler** that splits a checked plan across two arms without trading validity for speed (§3.4);
- a **demand-to-data bidder** that prices unserved gaps per demonstration episode (§3.5);
- an **evaluation** of four planners with per-round repair traces, a taxonomy of the failures caught, run-to-run stability, schedule speed-ups and the open-data coverage of the compiled plans (§4–§5).

## 2 Related Work

**Language models as planners.** Language models can decompose instructions into steps [11], ground them in skill affordances [1], close the loop with environment feedback [12], emit programs over robot APIs [17, 23], or compose spatial value maps for low-level motion [13]. Benchmarks show their plans are frequently invalid when checked formally [24].

**Planners with verifiers.** LLM+P translates problems to PDDL [20] for a classical planner [19]; LLM-Modulo pairs the model with external critics [14]; Text2Motion checks geometric feasibility of skill sequences [18]; task and motion planning integrates both levels [7]. We use the critic as a *repair signal* rather than a search engine, because the catalog is open-ended and a plan must also name, precisely, what is missing. Our operators follow STRIPS [6], extended with hand-scoped facts.

**Skills and policies.** Skills in our catalog are policies such as VLA models fine-tuned from demonstrations [2, 3, 16, 21], including bimanual ones from low-cost teleoperation [26].

**Demonstration data.** Robot demonstrations come from teleoperation corpora [4, 15, 22]; human demonstrations from egocentric video [5, 8–10]. We use both as candidate training sources for missing skills, and measure which of them the compiled plans actually point to.

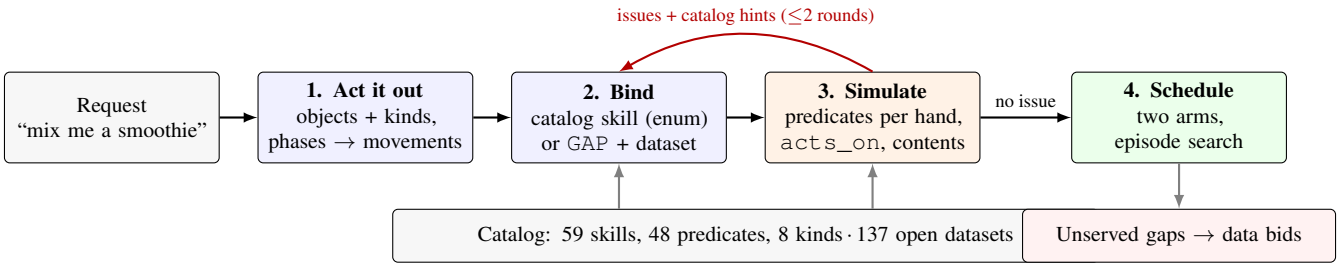



**Figure 1:** SKILLSTORE compiles a request in two model calls, checks the result with a declarative simulator whose complaints, with catalog-derived hints, go back to the binder, and schedules the valid plan on two arms. Only the binder sees the catalog. Plans that cannot run feed the data bidder.

# 3 Method

## 3.1 Skills as operators

The catalog holds 59 skills (11 trained, 5 beta, 43 listed without a policy). Each declares preconditions, postconditions and deletes over 48 predicates. Ten predicates are *hand-scoped* (`holding`, `hand_empty`, `object_located`, ...): `holding@left` and `holding@right` are distinct facts, which is what lets two sub-chains run on two arms. 8 skills declare which of 8 object kinds they *act on* (a bottle grasp acts on `bottle`, an item grasp on `item`); pouring skills declare a flow (held → target). Kinds are coarse on purpose—`bottle`, `glass`, `shaker`, `scoop`, `cloth`, `item`, `fixed`, `substance`—because they describe what a gripper closes around, not what an object is called; the full list is in Appendix A.

## 3.2 Two-stage compilation

**Act it out.** Without the catalog in view, the model writes the request as (i) the objects involved, each classified into an object kind, (ii) phases named by their outcome (“the glass holds juice”), each broken into movements that start with a movement verb (pick up, carry, tilt, press, drag), and (iii) a goal. Outcome verbs (make, operate, prepare) may name a phase but never a movement. A short start-state description fixes everyday assumptions (bottles open, appliances on their base and operated with a free hand). Keeping the catalog out of this stage matters: a model that plans *from* the catalog bends the task to fit it, and whatever the catalog cannot do is dropped or squashed into one gap the size of the request.

**Bind.** With the catalog and the open-dataset list in view, a second call maps movements to skills or to `GAP` entries. Structured outputs restrict skill identifiers to an enum of real skills, targets to the object names from stage one, and dataset identifiers to open datasets, so a hallucinated skill is impossible by construction. A gap is named as a reusable motion primitive (`twist_open`, `press_to_surface`) with conditions in the catalog’s vocabulary; its position in the plan is exact because gaps are emitted inline with the steps. Fig. 2 shows a compiled plan.

## 3.3 Simulation and repair

The simulator executes the plan step by step, gaps included with the effects they declare, and assigns hands by grouping steps into *episodes* (from the first step that leaves something on a hand to the step that frees it); overlapping episodes go to opposite hands. It raises *blocking* issues—unmet precondition, wrong object kind (`acts_on` $\not\ni$ kind of target), wrong pour direction (from a ledger of which object each hand holds and what each container contains), embodiment mismatch—and *advisory* ones: a plan ending with a hand still holding something, or a *pointless move* (an object picked up and set back where it was without being used). Issues are returned to the binder with hints derived from the catalog (“`positioned` is established by `move_to_station`”; “what holds something now: blender”). The loop runs at most two rounds (plus one for a pour in the wrong direction), and only the binding is retried: the acted-out script is not in question. Every number in this paper counts a plan as *valid* only if the simulator raises no blocking issue. Appendix B lists every check.

*“You have a paint bucket and brush. Draw an image on the table.”*
Objects: paint bucket `item`, paint `substance`,
paintbrush `item`, table `fixed`

1 `locate_object`(paintbrush)
2 `grasp_object`(paintbrush)
3 `move_to_station`(paint bucket)
`GAP dip_into` → EPIC-KITCHENS-100
4 `move_to_station`(paint bucket)
5 `move_to_station`(table)
`GAP drag_along_surface` → EPIC-KITCHENS-100
6 `move_to_station`(paint bucket)
7 `release_object`

Goal: an image is painted on the table and both hands are empty.

**Figure 2:** A compiled plan, verbatim from the Claude Opus 5.5 run (valid on the first binding). The grasp is an *item* grasp because stage one classified the brush as an item; the painting itself is two reusable gaps rather than one “draw image” step. Step 4 repeats step 3, an inefficiency the simulator does not flag because the brush is used in between.

## 3.4 Two-arm scheduling

A valid plan is turned into a dependency graph with four kinds of edge, all physical: *requires* (a predicate an earlier step established), *consumes* (clearing a predicate an earlier step established), *hand* (the same hand is still busy) and *resource* (two steps write the same world fact, or two arms want the same place—only one bottle fits over the glass). Steps are layered by their longest dependency chain; the number of layers is the schedule’s depth. Hands are chosen per episode, not per step, because a hand has to finish what it started: we search all assignments exactly up to 13 episodes and hill-climb

beyond, pin the first episode to the left hand (mirror schedules are equivalent), and reject any assignment that adds a blocking issue.

### 3.5 From gaps to data bids

Each request $r$ whose plan cannot run is logged against its blockers $B_r$ (gaps and untrained skills). For a blocker $s$, the $k$-th identical prompt counts $1/k$, and requests fade with age $a_r$ at half-life $h$:

$$w_{r,s} = \frac{2^{-a_r/h}}{k_{r,s}}, \qquad V_s = \sum_{r \ni s} w_{r,s} \frac{P_r}{|B_r|}, \tag{1}$$

where $P_r$ is the price of the whole plan (catalog prices, each gap at the catalog's median price), shared equally among its blockers because all of them must be filled. With $H$ future sales per logged request, target return $\rho$, a cap $c$ and $E_s$ episodes needed ($E_0$, reduced by a factor $1-\delta$ when an open dataset matches), the bid per episode and the budget pacing factor are

$$b_s = \min\Big(c,\ \frac{H\,V_s}{\rho\,E_s}\Big), \qquad \alpha = \min\Big(1,\ \frac{\text{budget}}{\sum_s b_s E_s}\Big), \tag{2}$$

and the posted bid is $\alpha\, b_s$. Pacing scales all bids equally, so more unmet demand always means a higher bid. Defaults: budget 5,000 EUR per month, $\rho = 3$, $H = 10$, $E_0 = 50$, $\delta = 0.4$, $h = 30$ days, $c = 60$ EUR.

## 4 Experimental Setup

**Requests.** Five demo requests (piña colada, juice, smoothie, wiping, clearing up) and eight unfamiliar ones (painting with a brush, a sandcastle, a card trick, watering plants, writing on a whiteboard, a jam sandwich, flowers in a vase, feeding a cat); the exact wording is in Appendix A. **Planners.** GPT-4.1 mini, GPT-4.1, Claude Sonnet 5.5 and Claude Opus 5.5 (effort *low*, standard speed), the same model for both stages and for repair. **Protocol.** Each request is compiled 3 times per planner with the compile cache off; the compiler records the simulator's issues at the start of every repair round, so a single run yields validity after 0, 1 and 2 rounds. We report proportions with 95% Wilson intervals [25]; 1 compilation that failed on a provider rate limit is excluded.

**Data.** 138 datasets are registered (137 openly accessible: 129 robot teleoperation, 6 egocentric human video, 3 simulation). 165 curated links connect each catalog skill to teleoperation datasets whose task text matches it, 317,112 episodes in total (median 443 per skill, minimum 50; Table 1). Every link was checked for licence, access, episode count and task text.

## 5 Results

### 5.1 Simulator feedback does most of the work

Without feedback the planners differ widely: 56% of GPT-4.1 mini's first bindings pass the simulator, against 95% for Claude Opus 5.5 (Table 2, Fig. 3). One round of repair closes most of that difference: GPT-4.1, Sonnet 5.5 and Opus 5.5 reach 97%, 100% and 100%, and GPT-4.1 mini rises to 85%. The second round adds little. With the checker in the loop,

**Table 1:** Catalog by category: skills, trained skills, and the median number of curated open teleoperation episodes per skill.

| Category | Skills | Trained | Median episodes |
|---|---|---|---|
| Cleaning | 6 | 1 | 480 |
| Finishing | 5 | 0 | 398 |
| Household | 16 | 0 | 428 |
| Manipulation | 13 | 4 | 444 |
| Mixing | 5 | 1 | 309 |
| Perception | 5 | 2 | 950 |
| Pouring | 4 | 1 | 618 |
| Transport | 5 | 2 | 1,141 |

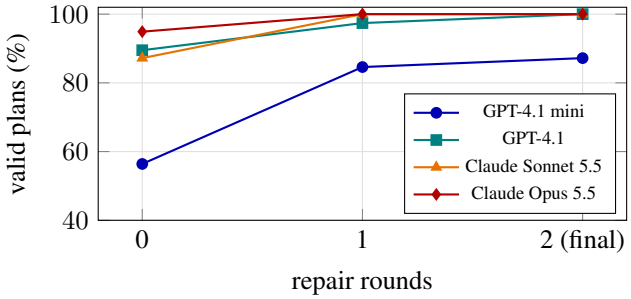



**Figure 3:** Share of plans the simulator accepts after each round of feedback.

GPT-4.1 and Sonnet 5.5 match Opus 5.5's final validity at a fraction of its latency (median 6 s and 9 s against 24 s per compilation). Beyond the smallest planner, the checker rather than model scale makes the plans installable; scale mostly buys fewer repair calls.

### 5.2 What the simulator catches

Of the 29 blocking issues raised on first bindings, 25 are unmet preconditions (Table 3), spread across requests—for instance a locate on a hand that is still holding something, or a release without a preceding carry. The remaining 3 wrong pour directions and 1 wrong object kind satisfy every predicate and would pass a precondition-only checker; they are caught only by the contents ledger and the `acts_on` declarations. After repair, 5 of 155 plans stay blocked, all from GPT-4.1 mini (a sandcastle, watering plants by pouring from an empty glass, and a smoothie). Advisory *pointless moves* halve with feedback (12→6) but persist, as the binder sometimes keeps a carry the first stage asked for.

### 5.3 Which requests are hard

Difficulty is concentrated (Table 4). Writing on a whiteboard, the jam sandwich, flowers in a vase and a glass of juice pass on the first binding for every planner and run. The sandcastle—two tools, a substance and a tray—is the only request no planner gets right every time on the first binding, and the smoothie trips three of four planners at least once. GPT-4.1 mini fails the first binding of the card trick, the piña colada and the table wiping in every run, yet repair fixes all of them; its remaining failures are the requests that need two tools or a pour.

**Table 2:** Planner comparison over 13 requests × 3 runs. Valid after $k$ repair rounds (%), final validity with 95% Wilson interval, plans ending with both hands empty (%), mean steps and gaps per plan, median seconds per compile.

| Planner | $k$=0 | $k$=1 | Final | Empty | Steps | Gaps | s |
|---|---|---|---|---|---|---|---|
| GPT-4.1 mini | 56 | 85 | **87** [73,94] | 95 | 11.5 | 0.8 | 7 |
| GPT-4.1 | 89 | 97 | **100** [91,100] | 100 | 10.8 | 1.7 | 6 |
| Claude Sonnet 5.5 | 87 | 100 | **100** [91,100] | 100 | 10.5 | 1.3 | 9 |
| Claude Opus 5.5 | 95 | 100 | **100** [91,100] | 100 | 13.4 | 1.8 | 24 |

**Table 3:** Issues raised at the first binding and on the final plan, pooled over all 155 compilations (plans affected, %).

| | First binding | | Final | |
|---|---|---|---|---|
| Issue | $n$ | % | $n$ | % |
| Unmet precondition | 25 | 16 | 3 | 2 |
| Wrong object kind | 1 | 1 | 0 | 0 |
| Wrong pour direction | 3 | 2 | 2 | 1 |
| Ends holding (advisory) | 5 | 3 | 2 | 1 |
| Pointless move (advisory) | 12 | 8 | 6 | 4 |

**Table 4:** First bindings the simulator accepts, per request and planner (out of 3; shading by share). †: not every run valid after repair. N: unfamiliar, D: demo request.

| Request | | 4.1 mini | 4.1 | Sonnet | Opus |
|---|---|---|---|---|---|
| Paint an image | N | 2/3 | 3/3 | 3/3 | 3/3 |
| Sandcastle | N | 0/3† | 2/3 | 2/3 | 2/3 |
| Card trick | N | 0/3 | 3/3 | 3/3 | 3/3 |
| Water plants | N | 1/3† | 0/3 | 3/3 | 3/3 |
| Write on whiteboard | N | 3/3 | 3/3 | 3/3 | 3/3 |
| Jam sandwich | N | 3/3 | 3/3 | 3/3 | 3/3 |
| Flowers in vase | N | 3/3 | 3/3 | 3/3 | 3/3 |
| Feed the cat | N | 3/3 | 3/3 | 2/3 | 3/3 |
| Piña colada | D | 0/3 | 3/3 | 2/3 | 3/3 |
| Glass of juice | D | 3/3 | 3/3 | 3/3 | 3/3 |
| Smoothie | D | 1/3† | 3/3 | 1/3 | 2/3 |
| Wipe table | D | 0/3 | 3/3 | 3/3 | 3/3 |
| Clear table | D | 3/3 | 2/2 | 3/3 | 3/3 |

### 5.4 Plans are similar, not identical

Runs of the same request produce similar plans—mean skill-sequence similarity between 0.85 and 0.93 (Table 5)—but rarely the same plan: only 23–38% of run pairs are identical. Differences are mostly in targets (which object is carried where) and in how a missing motion is split into gaps. For a store, this means a price quoted for a request is a property of one compilation, not of the request; caching compiled plans, as SKILLSTORE does, makes quotes reproducible.

### 5.5 Two arms help once hands are searched

With the simulator's default hand assignment—each new episode on the first free hand—only 25% of the 150 valid plans run anything in parallel (mean speed-up 1.13×, Fig. 4): the compiler is told to use one hand at a time unless two are needed, which keeps plans valid but serial. Searching hand assignments per episode raises this to 62% of plans, a mean speed-up of 1.37× (up to 2.0×), and 29% fewer time slots overall, without adding a single blocking issue. Parallelism is available in most household requests; it has to be found after validation, not asked of the model.

**Table 5:** Stability across the 3 runs of a request: mean pairwise similarity (1 − normalised edit distance) of skill sequences, with targets, share of identical plans, and Jaccard overlap of gap verbs.

| Planner | Skills | +Targets | Identical (%) | Gaps |
|---|---|---|---|---|
| GPT-4.1 mini | 0.86 | 0.74 | 23 | 0.84 |
| GPT-4.1 | 0.93 | 0.84 | 38 | 0.87 |
| Claude Sonnet 5.5 | 0.85 | 0.70 | 28 | 0.68 |
| Claude Opus 5.5 | 0.90 | 0.84 | 31 | 0.82 |

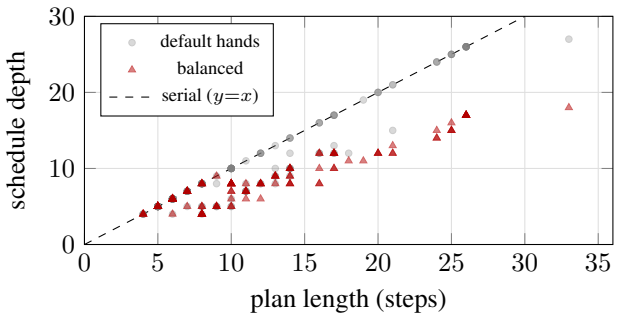



**Figure 4:** Schedule depth against plan length for all valid plans, with the default hand assignment and after the episode search. Points below the diagonal run steps on both arms at once.

### 5.6 Open-data coverage

Across all final plans, 87% of steps bind to trained skills, 2% to listed but untrained skills, and 11% are gaps. Of 218 gaps, 99% were matched to an open dataset—but 75% of those matches point to human egocentric video (Table 6) rather than robot teleoperation. The motions the catalog lacks (dipping a brush, spreading, scooping, twisting a lid open) are exactly those without robot demonstrations; human video can pretrain them, but not replace robot episodes. The catalog's own skills, by contrast, are well covered by teleoperation: every skill has at least 50 curated episodes (Table 1).

### 5.7 Demand needs canonical names

Gap names are not canonical: the 218 gaps carry 91 distinct names, and dipping a tool into a container alone appears under eight (Table 7). Grouping names by their leading verb collapses them to 27 motions. This matters for the bidder: treating each of the 155 compilations as a customer request, the market splits demand over 99 distinct blockers with raw names but 35 after grouping, and the top bid per episode rises from 10.41 to 20.21 EUR (Table 8): fragmented names hide the most wanted motions. Verb grouping is a blunt instrument—it merges pressing a button with pressing a brush onto a surface—so a

**Table 6:** Open datasets most often proposed for gaps (gap count).

| Dataset | Gaps |
|---|---|
| EPIC-KITCHENS-100 | 84 |
| EgoDex | 55 |
| Egocentric-10K | 12 |
| Ego-Exo4D | 11 |
| Wipe whiteboard (villekuosmanen) | 10 |

**Table 7:** Most frequent gap verbs: gaps, distinct names, examples.

| Verb | Gaps | Names | Examples |
|---|---|---|---|
| `drag` | 31 | 9 | `drag_on_surface`, `drag_along_surface` |
| `press` | 31 | 7 | `press_into`, `press_button` |
| `tilt` | 16 | 3 | `tilt_out`, `tilt_out_food` |
| `dip` | 15 | 8 | `dip_into`, `dip_into_vessel` |
| `scoop` | 15 | 7 | `scoop_item`, `scoop_material` |
| `peel` | 15 | 3 | `peel_off`, `peel_item` |
| `flip` | 15 | 3 | `flip_item`, `flip_over` |
| `twist` | 13 | 5 | `twist_item`, `twist_open` |

deployed store needs a proper motion taxonomy.

## 6 Discussion

**A checker substitutes for scale.** The largest planner is the most reliable on its own, but with two rounds of simulator feedback, planners that are roughly three times faster reach the same final validity. For an interactive store this is the right trade: the checker is cheap and exact, the model is slow and approximate, and the store pays the model's latency on every request.

**The data gap is the dexterity gap.** The catalog's transport and grasping skills have hundreds to tens of thousands of open robot episodes; the motions that compiled plans actually miss—dipping, spreading, peeling, twisting—have almost none and are matched to human video instead. Open robot data is broad in tasks but shallow in dexterous tool use, which is where demand concentrates.

**Safety is a property of the pipeline.** Because the compiler can be wrong, the gates are placed after it: installation and dispatch re-run the simulator on the current catalog rather than trusting a stored verdict, a queued plan cannot be edited, a runner claims a job atomically so two arms never receive the same plan, a plan whose runner died is failed rather than silently re-run, and an operator stop never sends the arm home. None of these depend on the model behaving well.

**From valid to certified.** A store should eventually publish, per skill and per robot, a measured success rate with a confidence interval; with 20 trials, 19/20 is 95% with a Wilson interval of roughly 76–99%, which is why our runner logs every attempt, failures included. Validity is the necessary first gate; measured success is the product.

**Table 8:** Simulated market: top bids per episode (EUR) with raw gap names and with names grouped by verb. Every compilation counts as one request; the default parameters of §3.5 apply.

| | Raw name | Bid | Grouped | Bid |
|---|---|---|---|---|
| 1 | `press_to_surface` | 10.41 | `press` | 20.21 |
| 2 | `tilt_out` | 7.39 | `drag` | 14.45 |
| 3 | `run_blender` | 6.54 | `tilt` | 10.74 |
| 4 | `drag_along_surface` | 6.28 | `twist` | 10.40 |
| 5 | `peel_item` | 4.96 | `peel` | 9.94 |
| 6 | `twist_item` | 3.86 | `run_blender` | 8.79 |

## 7 Limitations

**Valid is not successful.** The simulator checks what the catalog declares; a plan it accepts can still fail on hardware, and skills' success rates in our system so far come from a mock executor, so we report none. **The requests are not a held-out test set.** The prompts and start-state assumptions were revised while inspecting failures on these same requests (e.g. the appliance assumption after a smoothie plan held the blender jar while running it), so the numbers measure the final system on its development set. **Small sample.** Thirteen requests and 3 runs per planner give wide intervals; differences of a few points between planners are not significant. **Matching is declared, not demonstrated.** Dataset links rest on task-text matches and, for gaps, on the model's choice; we have not trained skills from them. **The market is simulated.** Bids in Table 8 come from our own compilations, where the same request recurs twelve times and is damped accordingly; real demand will differ. **Scheduling ignores gaps' timing.** Speed-ups are computed over catalog steps only, since gaps have no duration until they are trained.

## 8 Conclusion

Separating *acting out* a request from *binding* it to products, and letting a declarative simulator veto and explain, turns language models into usable compilers for a skill store: plans become checkable, schedules exploit both arms once hands are searched, gaps become atomic products with candidate data, and failures become named, countable categories. The next steps are closing the loop on hardware—replacing "valid" with measured success—canonicalising gap names into a motion taxonomy, and evaluating on requests the system has never seen.

## References


[1] M. Ahn, A. Brohan, N. Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[2] K. Black, N. Brown, D. Driess, et al. $\pi_0$: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[3] A. Brohan, N. Brown, J. Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.

[4] R. Cadene, S. Alibert, A. Soare, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. `https://github.com/huggingface/lerobot`, 2024.

[5] D. Damen, H. Doughty, G. M. Farinella, et al. Rescaling egocentric vision: Collection, pipeline and challenges for EPIC-KITCHENS-100. *International Journal of Computer Vision*, 130:33–55, 2022.

[6] R. E. Fikes and N. J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.

[7] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021.

[8] K. Grauman, A. Westbury, E. Byrne, et al. Ego4D: Around the world in 3,000 hours of egocentric video. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[9] K. Grauman, A. Westbury, L. Torresani, et al. Ego-Exo4D: Understanding skilled human activity from first- and third-person perspectives. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[10] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang. EgoDex: Learning dexterous manipulation from large-scale egocentric video. *arXiv preprint arXiv:2505.11709*, 2025.

[11] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International Conference on Machine Learning (ICML)*, 2022.

[12] W. Huang, F. Xia, T. Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2022.

[13] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. VoxPoser: Composable 3d value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.

[14] S. Kambhampati, K. Valmeekam, L. Guan, et al. Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks. In *International Conference on Machine Learning (ICML)*, 2024.

[15] A. Khazatsky, K. Pertsch, S. Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024.

[16] M. J. Kim, K. Pertsch, S. Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.

[17] J. Liang, W. Huang, F. Xia, et al. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[18] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg. Text2Motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47:1345–1365, 2023.

[19] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

[20] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control, 1998.

[21] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.

[22] Open X-Embodiment Collaboration. Open X-embodiment: Robotic learning datasets and RT-X models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[23] I. Singh, V. Blukis, A. Mousavian, et al. ProgPrompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[24] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[25] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[26] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023.

## A Requests and object kinds

**Unfamiliar requests (N).** "You have a paint bucket and brush. draw an image on the table"; "build a small sandcastle on the tray with the bucket and shovel"; "do a simple card trick with the deck on the table"; "water the two plants on the shelf"; "write my name on the whiteboard with the marker"; "make me a jam sandwich"; "put the flowers in the vase and add water"; "feed the cat: open the tin and put the food in her bowl". **Demo requests (D).** "make me a pina colada"; "pour me a glass of juice"; "mix me a smoothie"; "wipe my table clean after dinner"; "clear the table and put the bottles away".

**Object kinds.** `bottle`: a rigid bottle, carton or jug of liquid, held upright; `glass`: a drinking glass, cup or mug; `shaker`: a cocktail shaker; `scoop`: a measuring scoop with a handle; `cloth`: something soft and deformable; `item`: any other rigid, hand-sized loose thing; `fixed`: something that is not picked up (a table, a shelf, an appliance); `substance`: a material inside something else (paint, water, sand).

## B Simulator checks

| Check | Raised when |
|---|---|
| Unmet precondition | a step needs a fact, on its hand, that no earlier step or gap established |
| Wrong object kind | the skill's `acts_on` does not include the kind of its target |
| Wrong pour direction | the held object is the target, or is an empty glass or shaker |
| Embodiment mismatch | the skill needs a body (torso, base) the robot lacks |
| Unknown skill | a step names a skill outside the catalog (prevented by the enum) |
| Empty plan | the binding contains no executable step |
| Ends holding *(adv.)* | a hand still holds or has located something at the end |
| Pointless move *(adv.)* | an object is picked up and set back where it was, unused |

**Table 9:** Every check the simulator runs; *adv.*: advisory, returned to the binder but not blocking installation.

**Repair hints.** Each issue returns with a catalog-derived hint: the skills that establish a missing fact, the containers holding something, or the skill a duplicate gap restates.